

Interoperabilidad

MANUAL DE CONFIGURACIÓN NODO v2

PLATAFORMA
INTEGRADA DE
SERVICIOS
ELECTRÓNICOS DEL
ESTADO



Interoperabilidad descentralizada, flexible y ágil

Versión 2.0.4 | Febrero 2024

ÍNDICE

1. Guía Rápida	3
2. Contexto	4
3. Modelo PISEE 2	4
3.1. ¿Cómo funciona el modelo?	4
3.2. ¿Qué es el catálogo de servicios?	4
3.3. ¿Qué es el registro de trazabilidad?	5
4. Uso del Nodo	6
4.1. Características del Nodo	7
4.1.1. ¿Que es el Nodo de Interoperabilidad de PISEE 2?	7
4.1.2. ¿Qué contiene?	7
4.1.3. ¿En qué sistema operativo funciona el Nodo?	7
4.1.4. ¿Cómo se ejecuta y administra?	7
4.2. ¿De qué forma le sirve el Nodo a los organismos?	8
4.2.1. Como kit de conexión a PISEE 1	8
4.2.2. Como consumidor de servicios (Contra otro Nodo DGD)	8
4.2.3. Como consumidor de servicios (Otro tipo de nodos)	9
4.2.4. Como proveedor de servicios	9
4.3. Requisitos del Nodo	11
4.3.1. Información de Puertos	12
4.4. Instalación del Nodo	13
4.4.1. Obtención del Nodo	13
4.4.2. Instalación en un equipo	13
4.4.2.A. Instalación en Linux	13
4.4.2.B. Instalación en Windows	15
4.5. Configuración del Nodo	16
4.5.1. Archivo de configuración	16
4.5.2. Agregar servicios para consumir web services	16
4.5.3. Agregar servicios para proveer un web service a otros organismos	18
4.5.4. Configurar credenciales	20
4.5.5. Configuración de los servidores internos del Nodo	23
Ejemplo:	24
4.5.6. Configuración de la conexión al servidor central	25
4.5.7. Configuración de la base de datos local	26
4.5.8. Configuración del Log	28
4.6. Comandos del Nodo	29

Tabla 1 - Resumen de comandos nodo	34
5. Conversión de mensajes	35
5.1. Concepto	35
5.2. Cómo convertir mensajes para un servicio	36
5.2.1. Asignar un script al mensaje de consulta	36
5.2.2. Asignar un script al mensaje de respuesta	36
5.3. Cómo escribir los script de conversión	37
6. Interfaces de consulta	42
6.1. Concepto	42
6.2. Cómo utilizar las interfaces	43
6.3. Cómo asignar una interfaz para un servicio	45
6.4. Cómo escribir una interfaz	45
7. Consideraciones Generales	47
7.1. Transacciones	47
7.2. Política de actualizaciones	47
8. Anexos	48
8.1. Obtener llaves pública y privada desde un .pfx	48
8.2. DockerFile de Ejemplo	48

INFORMACIÓN DOCUMENTO

AUTOR DOCUMENTO	Juan Carlos Daille, Rodrigo Inostroza, Francisco De la Carrera
FECHA ACTUALIZACIÓN	08 DE NOVIEMBRE DE 2023
CONTRAPARTE INSTITUCIÓN	pisee@digital.gob.cl
NÚMERO DE VERSIÓN	v2.0 - Actualización de funcionalidades nuevas nodo v2 v2.0.1 - Aclaración sobre encriptación y privacidad de transmisión de mensajes v2.0.2 - Agregado dockerfile en anexos v2.0.3 - Agregar servicios para proveer un web service a otros organismos v2.0.4 - Mejoras en la funcionalidad de transferencia de grandes archivos

I. Guía Rápida

- 1) Descomprima el contenido de `Nodo.zip` en una carpeta.
- 2) Abra una consola de terminal y, dentro de la carpeta donde se encuentra el nodo, ejecute:
`./NodoV2 start` (o `NodoV2.exe start` en windows)
- 3) Cuando el Nodo ya se haya iniciado abra otra terminal y ejecute el comando:

```
curl -X GET http://localhost:8084/detalleOrganismo/PE-SUB-00053
```

- 4) Este comando ejecutará una consulta web a su Nodo, el Nodo a su vez consultará, a través de internet, a un Nodo de prueba de la SGD y este le responderá su consulta de prueba.
- 5) Si este comando le retorna datos similares a estos significa que su Nodo funciona correctamente:

```
{
  "id": 53,
  "codigo": "PE-SUB-00053",
  "descripcion": "Subsecretaría de Hacienda",
  "fecha_inicio": "2021-12-14T11:39:58.649389Z",
  "fecha_fin": "2999-12-30T21:00:00-03:00",
  "IdCodificacion": 1,
  "info_extra": "",
  "padre": 537,
  "id_codigo_padre_jerarquico": 0,
  "secuencia": 53,
  "fecha_cambio": "0001-01-01T00:00:00Z",
  "identificador": "1.PE-SUB-00053",
  "vigente": true
}
```

- 6) Estando su Nodo funcionando ya puede agregar servicios para poder consumir datos o exponer sus propios servicios en PISEE 2 con seguridad.

2. Contexto

Todo lo relacionado a PISEE 2 deriva de la implementación de la [Ley de Transformación del Estado 21.180](#) y es parte de las [plataformas transversales de Gobierno Digital](#).

3. Modelo PISEE 2

3.1. ¿Cómo funciona el modelo?

PISEE 2 es una plataforma de interoperabilidad donde el consumidor y el proveedor de un servicio web se comunican directamente, sin ningún organismo intermediario, para intercambiar información pero utilizando, tanto en el consumidor como en el proveedor, una software llamado Nodo que funciona como la puerta de entrada y salida de mensajes. El Nodo cumple las funciones de simplificar la tarea del consumidor al mismo tiempo que brinda una fuerte capa de ciberseguridad al proveedor del servicio web.

En PISEE 2 si dos organismos requieren interoperar deben instalar ambos Nodos de interoperabilidad. El proveedor debe exponer sus servicios web a través de su Nodo y publicarlo en el catálogo de servicios luego el consumidor llamará a dichos servicios a través de su propio Nodo, descansando ambos organismos en que el Nodo ya realiza el trabajo de autenticación, autorización, encriptación, firma digital de los mensajes y del registro de la trazabilidad de cada transacción.

Al estar los Nodos instalados en los dos organismos que desean interoperar, la comunicación es de punto a punto y por lo tanto, los datos que se transmiten a través de los Nodos **no pasan nunca por otra entidad u otra plataforma**, resguardando así la confidencialidad de los datos transmitidos.

3.2. ¿Qué es el catálogo de servicios?

El catálogo de servicios del Estado es un servicio centralizado que posee el listado de todos los servicios webs proporcionados por los organismos de la administración del Estado y los endpoints de estos servicios webs.

Los Nodos deben conectarse a este catálogo para descubrir la información técnica de los servicios web (endpoint, qué protocolo se debe utilizar, metadata, etc.)

3.3. ¿Qué es el registro de trazabilidad?

PISEE 2 posee un registro de todos los intercambios de datos entre organismos de la administración del Estado. Este registro **solo mantiene la metadata** de las transacciones de información efectuadas y **nunca los datos intercambiados**.

En PISEE 2 cada vez que un organismo se conecta a otro se genera un identificador único de la transacción (ID Transacción) el cual es una cadena de caracteres que se compone del código del organismo consumidor del servicio y un ULID aleatorio. Ejemplo: `6.01GK313E4VSGGB69DKNPTFAM6K`
(Especificación de ULID <https://github.com/ulid/spec>)

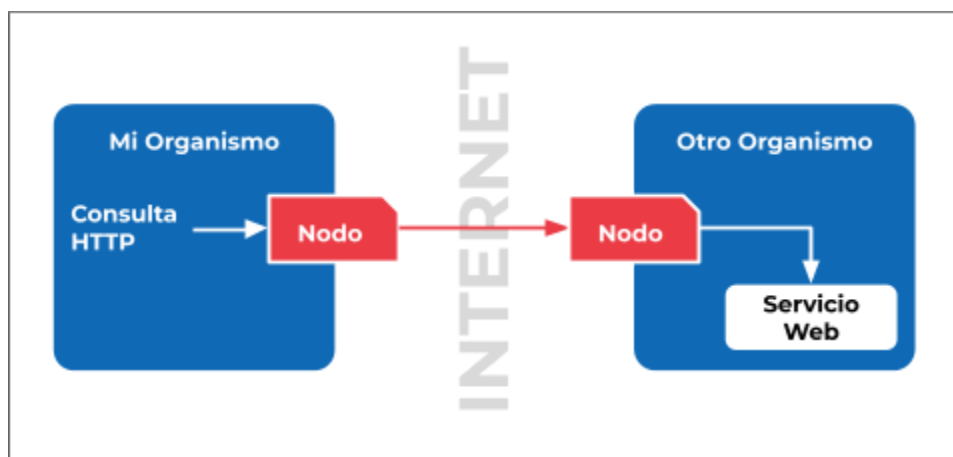
Una transacción se compone de 4 etapas: Envío de un mensaje por parte del consumidor de un servicio web, recepción del mensaje por parte del proveedor del servicio web, respuesta del proveedor del servicio web y recepción de la respuesta por parte del consumidor del servicio web. El registro de trazabilidad almacena estos cuatro registros, siempre asociados al ID de transacción.

4. Uso del Nodo

El Nodo es un software que se instala localmente en cada organismo y que permite la circulación de mensajes de interoperabilidad tanto de entrada como de salida, asegurando el cumplimiento de la norma técnica de interoperabilidad y la Ley de Transformación Digital.

No interfiere en los mensajes entre organismos ni condiciona sus formatos ya que opera como un proxy seguro entre el exterior y sus servicios web y aplicaciones, siempre compartiendo información de operación (trazabilidad, certificados, alertas) de manera automática y transparente con los servicios centralizados de apoyo a la interoperabilidad del modelo PISEE 2.

El Nodo es la herramienta que busca facilitar la interoperabilidad entre organismos simplificando y asegurando la comunicación.



Esquema de comunicación entre nodos

4.1. Características del Nodo

4.1.1. ¿Qué es el Nodo de Interoperabilidad de PISEE 2?

El Nodo es una aplicación que permite a los organismos del Estado conectarse a la red de interoperabilidad PISEE 2.

Se instala en la infraestructura de cada organismo del Estado y es administrada por el personal de dichos organismos.

Es totalmente autocontenida y no requiere componentes adicionales.

4.1.2. ¿Qué contiene?

El Nodo consiste en una carpeta que contiene:

- El ejecutable del Nodo
- El archivo de configuración del Nodo
- Certificados de seguridad

4.1.3. ¿En qué sistema operativo funciona el Nodo?

Funciona tanto en Linux como en Windows, ya que se entrega compilado nativamente para cada sistema operativo. Solo es necesario que el sistema operativo sea de 64 bits.

4.1.4. ¿Cómo se ejecuta y administra?

Funciona mediante línea de comandos con los cuales se puede ejecutar, consultar información de su estado, servicios y seguridad.

Además cuenta con un archivo de configuración (`config.json`) donde se encuentra toda la información del comportamiento del nodo y los servicios que gestiona.

4.2. ¿De qué forma le sirve el Nodo a los organismos?

4.2.1. Como kit de conexión a PISEE I

El Nodo cumple además funciones como Kit de Conexión a PISEE 1, haciendo mucho más simple la integración a esta plataforma y permitiendo que esa integración sea también compatible con PISEE 2 cuando PISEE 1 sea deprecada (se dará de baja el 31 de diciembre del 2024).

El nodo recibe solo los datos necesarios para la consulta y se encarga de crear la estructura sobre XML y de crear el mensaje SOAP por sí mismo añadiendo además, para el caso de registro civil, aspectos como la firma y descriptación de los mensajes de entrada y salida con llaves asíncronas.

Este aspecto del nodo puede ser usado desde ya y no requiere ningún cambio de convenios ni acuerdos, ya que es solo un kit de integración a la plataforma actual de interoperabilidad.

4.2.2. Como consumidor de servicios (Contra otro Nodo DGD)

Gracias al Nodo es sencillo consumir servicios web del Estado registrados en la plataforma de interoperabilidad PISEE 2, cumpliendo además con todos los requerimientos de la ley de transformación digital del Estado y con las normas técnicas de interoperabilidad y de ciberseguridad.

Para consumir un servicio, al cual fue autorizado su organismo por el proveedor, solo es necesario agregar en la configuración del Nodo el nombre del servicio tal como aparece en el Catálogo de Servicios y el Nodo se encargará de descubrir la ruta real del servicio, de establecer los canales de comunicación seguros y de registrar la trazabilidad, todo esto sin tener que realizar cambios en los aplicativos propios.

Si el proveedor posee también un Nodo DGD se utilizarán los protocolos de autenticación, autorización y mensajería generales del modelo.

4.2.3. Como consumidor de servicios (Otro tipo de nodos)

En el caso de que el proveedor del servicio posea un Nodo que no sea el provisto por DGD este podría llegar a tener un protocolo de autenticación, autorización y de mensajería propio, pero este protocolo debió ser registrado previamente como un protocolo válido, descrito en la guía técnica y por lo tanto el Nodo DGD del consumidor ya tendrá la capacidad de entender las diferencias y de adaptarse a ellas sin que se requiera del organismo consumidor ninguna otra acción o configuración.

4.2.4. Como proveedor de servicios

El Nodo tiene la funcionalidad de exponer servicios web que estén dentro de la infraestructura del organismo en la red de interoperabilidad del Estado PISEE 2 para que puedan ser utilizados, de manera segura, por otros organismos del Estado previa autorización de parte del proveedor.

Exponer servicios a través del Nodo tiene una serie de ventajas:

- Coloca una estricta capa de seguridad que protege al servicio web en internet basada en autenticación mutua entre consumidor y proveedor, (Mutual TLS).
- Encriptación de punto a punto.
- Almacenamiento automático de la trazabilidad en el servidor central de PISEE 2
- Control del volumen de transacciones recibidas (TPS).
- Firma digital de los mensajes para evitar distorsiones del mensaje durante el trayecto y garantiza el origen del mensaje.
- No tiene restricciones sobre el tipo de datos ni estructura de los mensajes que se intercambian.
- No es necesario hacer **ningún cambio** en el servicio web original.

Para utilizar el Nodo para exponer un servicio web basta con:

- Registrar en el archivo de configuración del Nodo la ruta interna del servicio web y colocarle un nombre al servicio
- Registrar en el Servidor central de PISEE 2 el nombre de servicio y la ruta real que tendrá hacia internet.
- Solicitar un certificado de identidad del organismo para PISEE 2 con la División de Gobierno Digital.
- Dar permisos a un organismo del Estado en la misma plataforma.

Más detalles de la configuración del Nodo como proveedor se encuentra más adelante en la sección [4.5.3](#)

4.3. Requisitos del Nodo

El Nodo es una aplicación que funciona como Middleware entre sus aplicaciones y los servicios del Estado.

- Está compilada de manera nativa para Windows y Linux.
- No requiere servidores de aplicaciones ni librerías adicionales ni entornos especiales.
- La aplicación pesa aproximadamente 26 MB.
- No tiene requisitos específicos en cuanto a memoria ni procesador para ejecutarse (aunque esto sí impacta en su velocidad de procesamiento).
- El nodo requiere acceso a una carpeta donde pueda dejar archivos temporales propios de su operación (rutas y certificados públicos).
- Opcionalmente deja registros de log para su propia institución.
- Puede ser instalado las veces que quieran en su organización para obtener alta disponibilidad. Solo se debe cuidar que no choquen los puertos si se instalan en la misma máquina.
- Se puede instalar en Docker.
- La configuración del nodo se realiza modificando un archivo llamado `config.json` que se entrega preconfigurado, pero los organismos son libres de modificarlo.
- Para hacer funcionar el Nodo como **proveedor** de servicios, es necesario abrir un puerto hacia el exterior (por defecto es el **8489**).
- Es importante que **el puerto de uso interno no quede expuesto al exterior**.

- Además del puerto 8489, es necesario que habiliten las siguientes IP's / DNS, dependiendo del ambiente en el que se encuentre:

```
http://18.237.240.119:8500
```

```
http://18.237.240.119:8444
```

Dirección ambiente de desarrollo

```
catalogo.pisee.cl:8500
```

```
https://server.pisee.cl:8444
```

Dirección ambiente de producción

4.3.I. Información de Puertos

- **Catálogo de Servicios:**
puerto 8500, es un puerto de Salida
- **Servidor Central:**
Puerto 8444, es un puerto de Salida
- **Nodo como "Consumidor":**
Puerto 8084, es un puerto de Entrada de mensajes (No es necesario exponerlo a internet si las aplicaciones que consumen a través del Nodo están en el mismo servidor, en caso contrario sí habría que exponerlo)
- **Nodo como "Proveedor":**
Puerto 8489, es un puerto de Entrada de mensajes, pero de entrada de mensajes desde el Exterior de la Institución.

4.4. Instalación del Nodo

4.4.1. Obtención del Nodo

Obtenga la última versión del Nodo directo con el equipo PISEE, quienes le entregarán el Nodo preconfigurado con sus credenciales.

4.4.2. Instalación en un equipo

4.4.2.A. Instalación en Linux

Exclusivamente para los servicios de SRCeI que se encuentran en la antigua PISEE 1 se debe instalar la librería xmlsec1. Si aún no está instalado `xmlsec1`, escribir en la consola:

```
sudo apt-get update
sudo apt-get install xmlsec1
```

- Si es una distribución de RedHat:

```
yum install xmlsec1-openssl
sudo ln -s libxmlsec1-openssl.so.1 libxmlsec1-openssl.so
```

- Copiar el contenido del aplicativo en algún directorio de su infraestructura.
- Ejecutar `chmod` para darle permisos de ejecución al aplicativo:

```
sudo chmod +x ./Nodo
```

- Copiar su certificado en formato PEM (`cert.pem` y `key.pem`) a la carpeta `"/certsX509"` del aplicativo.
Luego modifiquen el archivo `config.json` en la sección `"X509"` para actualizar los nombres de los archivos.

```
"SRCeI": {
  "X509": {
    "privado": "./certsX509/keyDelOrganismo.pem",
```

```
"publico": "./certsX509/certDelOrganismo.pem"  
  }  
}
```

- Ejecutar el aplicativo:

```
sudo ./Nodo start &
```

Esto ejecutará la aplicación en background y levantará un servidor web para su uso interno en el puerto 8084 (Por defecto).

Dejar ejecutando la aplicación del nodo para habilitar el servicio. Si se cierra la app, se terminarán las conexiones (no se ejecuta en el fondo).

ATENCIÓN

No se debe dejar este puerto accesible para el exterior de su organización.

Luego ya pueden consumirlo desde sus aplicaciones o hacer un curl o postman a las rutas del nodo. (Estas rutas se les indicaron en el correo de descarga del nodo)

4.4.2.B. Instalación en Windows

Copiar el contenido del aplicativo en algún directorio de su infraestructura.

Copiar su certificado en formato PEM (`cert.pem` y `key.pem`) a la carpeta `/certsX509` del aplicativo.

ATENCIÓN

Si lo copian con esos mismos nombres no deben modificar el archivo de configuración, si los dejan con otros nombres deben editar el archivo `config.json`

Desde una consola ejecutar el aplicativo, usando el comando:

```
./Nodo.exe start
```

Esto ejecutará la aplicación y levantará un servidor web para su uso interno en el puerto 8084 (Por defecto)

ATENCIÓN

No se debe dejar este puerto accesible para el exterior de su organización.

4.5. Configuración del Nodo

4.5.1. Archivo de configuración

El Nodo posee un archivo de configuración llamado `config.json` que contiene toda la información relacionada al funcionamiento del Nodo. Este archivo se encuentra en la misma carpeta que el Nodo, está construido en formato JSON y sin él el Nodo no puede funcionar.

Cualquier cambio que se desee hacer para el Nodo se debe realizar en este archivo.

NOTA

Este archivo se puede editar en tiempo de ejecución del Nodo, el cual detectará los cambios y cargará nuevamente la configuración, pero en caso de que tenga errores graves de formato hará que el Nodo se detenga y deba ser reiniciado manualmente.

4.5.2. Agregar servicios para consumir web services

Para consumir un web services, al cual el organismo proveedor le otorgó previamente acceso, se debe realizar un cambio en el archivo de configuración.

Primero se debe buscar los detalles del servicio en el catálogo de servicios de PISEE 2.

Luego en el archivo `config.json` se debe agregar lo siguiente en la sección de `"Consumidor"`:

```
"consumidor": [  
  {  
    "nombre": "<NombreDelServicioSegunElCatalogo>",  
    "rutaLocal": "/algo/nombreInternoParaElServicio"  
  }  
]
```

Donde `"nombre"` corresponde al nombre de servicio tal cual aparece en el catálogo de servicios.

Y `"rutaLocal"` es la ruta que le permitirá llamar al servicio desde sus aplicaciones de la forma `/path` por ejemplo:

```
/algo/nombreInternoParaElServicio
```

Una vez modificado se podrá consultar al servicio llamando a:

```
http://ip_local_del_nodo:8084/algo/nombreInternoParaElServicio
```

Para el caso específico de querer consumir servicios que aún son parte de PISEE 1 es necesario agregar un par de valores "servicioPisee" y "tramitePisee".

```
"consumidor": [  
  {  
    "nombre": "<NombreDelServicioSegunElCatalogo>",  
    "rutaLocal": "/algo/nombreInternoParaElServicio",  
    "servicioPisee": "<Nombre servicio pisee 1>",  
    "tramitePisee": "<Nombre de tramite de pisee 1>"  
  }  
]
```

Desde la versión 2 del Nodo se incluyó la herramienta de convertir los mensajes mediante la ejecución de scripts personalizados por cada web service. Esto permite mantener la lógica de las aplicaciones intacta a pesar de que el formato de los mensajes cambie en el proveedor.

Para activar la ejecución de un script tanto de entrada como de respuesta basta agregar los tags al archivo config.json indicando el nombre de archivo del script.

```
"consumidor": [  
  {  
    "nombre": "<NombreDelServicioSegunElCatalogo>",  
    "rutaLocal": "/algo/nombreInternoParaElServicio",  
    "conversion": {  
      "entrada": "entrada.dummy.echo.script",  
      "respuesta": "salida.dummy.fake.script"  
    }  
  }  
]
```

Para más información de cómo obtener y crear script de conversión vaya a las secciones [5.2.1](#) y [5.2.2](#).

4.5.3. Agregar servicios para proveer un web service a otros organismos

Para exponer un servicio ya sea en desarrollo o producción, se deben realizar dos pasos, modificar el archivo de configuración y enviar información del servicio al equipo de interoperabilidad DGD.

El primero consiste en que deben agregar el archivo de configuración de la institución que desea proveer un servicio, en la sección "proveedor", donde se debe completar la siguiente estructura:

```
"proveedor": [  
  {  
    "nombre": "<NombreDelServicioSegunElCatalogo>",  
    "origen": {  
      "tipo": "WEB_SERVICE",  
      "rutaLocal": "http://ip/ruta_local/servicio"  
    },  
    "rutaExterna": "/nombreServicio",  
    "tps": <n>  
  }  
]
```

Donde:

- El "nombre" será el identificador del servicio en el catálogo con formato PascalCase (todo junto y mayúsculas en la primera letra de cada palabra unida).
- El "tipo" de "Origen" siempre debe ser **WEB_SERVICE**. En ese sentido, no les debe preocupar si su servicio es una API o similar, en este caso hace referencia a que es un aplicativo y no, por ejemplo, un excel.
- La "rutaLocal" de "Origen" corresponde a la URL real del servicio, puede ser una ruta localhost en caso de que el aplicativo esté en el mismo servidor que el Nodo.
- La "rutaExterna" es el path del servicio. Por ejemplo, si el valor puesto fuese /mtt/servicio1 entonces en el catálogo la ruta de ése endpoint sería:

```
https://<ruta_nodo>/<proveedor>/<valor_rutaExterna>
```

- Las "tps" corresponden al máximo de transacciones por segundo que la institución proveedora desea limitar para ese servicio.

La segunda parte del proceso, deben enviar al equipo de interoperabilidad al correo pisee@digital.gob.cl, la información que aparece listado a continuación. Con esta información el equipo de PISEE habilitará un Nodo consumidor, configurado para los servicios que se desean proveer, como una herramienta para hacer pruebas como un consumidor real.

Datos a enviar:

1. La IP o DNS del Nodo que va a exponer el servicio.
2. El puerto por el cual se va a exponer el Nodo hacia Internet (en el `config.json` está en la sección `puerto->externo`, por defecto 8489)
3. El valor que colocaron en `"rutaExterna"`
4. Un ejemplo de cómo llamar el servicio (si es con `path params` o un `POST` con body), esto es necesario para enviarlo como documentación a los consumidores al momento de habilitarlos.

Con esta información, el equipo de interoperabilidad registrará, en el Catálogo de Servicios de PISEE 2, el servicio con el mismo nombre y ruta externa, con el propósito que los nodos de los consumidores puedan encontrar y acceder al servicio web.

4.5.4. Configurar credenciales

El archivo de configuración posee la sección `"identificacion"` donde se colocan todas las credenciales necesarias para identificarse tanto en la red PISEE 2 como con los organismos que son contrapartes.

```
"identificacion": {  
  "pki": ...,  
  "firma": ...,  
  "credenciales": ...,  
  "SRCeI": ...,  
  "SII": ...  
}
```

`"pki"`: posee las rutas de las credenciales (llaves pública y privada) para autenticarse en la red PISEE 2.

```
"pki": {  
  "privado": "../key.pem",  
  "publico": "../cert.pem"  
}
```

`"firma"`: contiene la llave privada de firma de mensajes para los mensajes de PISEE 2 (Utilizado para el protocolo MPGA).

```
"firma": {  
  "privado": "4162b1b67407770c79e9d9a29c0e...",  
}
```

`"credenciales"`: contiene datos de identificación como:

- `"id"` es el código del organismo (según el gestor de códigos del estado [ver](#))
- `"nombrePISEE"` corresponde al nombre del organismo tal como estaba registrado en PISEE 1

- "credencialesPISEE" credencial del organismo para PISEE 1

```
"credenciales": {  
  "id": "PE-MIN-00529",  
  "nombrePISEE": "MTT",  
  "credencialesPISEE": "4162b1b6"  
}
```

Después es posible colocar credenciales específicas para organismos con protocolos distintos al estándar de PISEE 2 (MPGA)

- "SRCeI": Registro civil exige credenciales propias para la interoperabilidad, tanto para la API antigua (SOAP-XML) como para la nueva (REST-JSON). Estas credenciales deben ser previamente registradas en el Registro Civil.

- **"X509"**: posee las rutas de llaves privada y pública necesarias para la firma y descriptación requeridas por registro civil para servicios del tipo PISEE 1 (SOAP-XML). Estas rutas deben referirse a un archivo de llaves de tipo PEM
- **"oauth2"**: posee las credenciales de la API REST de registro civil.

```
"firma": {  
  /* exclusivo para servicios PISEE 1 */  
  "X509": {  
    "privado": "./certs/key.pem",  
    "publico": "./certs/cert.pem"  
  },  
  
  /* exclusivo para nueva API de SRCeI */  
  "oauth2": {  
    "clientId": "id_organismo",  
    "clientSecret": ".....",  
    "username": "nombre_usuario",  
    "password": "....."  
  }  
}
```

- **"SII"**: El servicio de impuestos internos también exige credenciales propias para la interoperabilidad, las cuales deben estar previamente registradas en dicho organismo. En el archivo de configuración se debe indicar donde se encuentran los archivos de las llaves pública y privada.

```
"SII": {  
  "privado": "./certs/sii/key.pem",  
  "publico": "./certs/sii/cert.pem"  
}
```

En el caso de que existan más organismos proveedores con credenciales específicas, estas deberán ir en esta misma sección.

- **"FirmaGob"**: La plataforma FirmaGob exige credenciales propias para la interoperabilidad, las cuales deben estar previamente registradas en dicha plataforma. En el archivo de configuración se debe indicar la entidad su api token key y el secret asociado.

```
"FirmaGob": {  
  "entity": "Nombre de la entidad",  
  "atk": "api token key",  
  "sk": "Alkas76askjhdass7123kjh"  
}
```

4.5.5. Configuración de los servidores internos del Nodo

En la sección de **"servidores"** se configuran los dos servidores del Nodo:

- El servidor interno recibe peticiones del mismo organismo para poder consumidor de servicios provistos por otros organismos.
- El servidor externo recibe peticiones de otros organismos que quieren consumir los servicios que su organismo provee.

```
1"servidores": {  
  "interno": {  
    "puerto": "8084",  
    "tps": 1000,  
    "backlog": 100,  
    "seguridad": {  
      "passwordHTTTPBasico": "alan:YnJpdG8="
```

Del servidor **interno** se configuran los siguientes valores:

- `"puerto"`: indica el puerto del servidor interno.
- `"tps"`: indica el valor máximo de transacciones por segundo que acepta el servidor interno.
- `"backlog"`: el backlog es una lista temporal que contiene el exceso de mensajes que sobrepasan las TPS y que los envía cuando ya baja la sobrecarga de mensajes.
- `"seguridad"`: en la sección de seguridad se configuran las herramientas de control de acceso del servidor local.
- `"passwordHTTPBasico"`: este parámetro obliga a que todo llamado local a los servicios del Nodo de interoperabilidad deban enviar un usuario y una contraseña que deben ser los mismos que se especifican en este campo.

El valor de este campo se compone de `<nombre>:<password>` y debe ser enviado en la cabecera `X-PISEE-LocalAuthorization`.

Ejemplo:

Si en su configuración tiene este valor:

```
"seguridad": {  
  "passwordHTTPBasico": "alan:YnJpdG8="  
}
```

Entonces los llamados para consumir servicios deben agregar la cabecera `X-PISEE-LocalAuthorization` con dicho valor, separando usuario y password con un "@":

```
curl -X POST http://localhost:8084/dummy/fake  
-H X-PISEE-LocalAuthorization:alan@YnJpdG8= -data '{"campo':  
'cualquier valor'}"
```

Del servidor **externo** se configura el siguiente valor:

"puerto": indica el puerto del servidor externo.

```
"externo": {
  "puerto": "8489"
}
```

NOTA

Este puerto **debe quedar expuesto a internet** para que los demás organismos puedan acceder a los servicios web publicados.

El servicio expuesto en este puerto es HTTPS.

4.5.6. Configuración de la conexión al servidor central

El Nodo requiere conectarse a los servicios centralizados que apoyan la interoperabilidad tanto para acceder al catálogo de servicios, al registro de trazabilidad y al almacén de certificados.

En esta sección se configuran las rutas, credenciales y certificados necesarios para conectarse a los servicios centralizados.

```
"servidorCentral": {
  "catalogo": {
    "host": "catalogo.pisee.cl:8500",
    "tls": {
      "publico": "dcX-client-consul-0.pem",
      "privado": "dcX-client-consul-0-key.pem"
    }
  },
  "jwtkey": "LS0tLS1CRUdJTjB.....",
  "autorizadorCertificadoPublico": "./public_autorizador.pem",
  "certificadoPublico": "./cacert.pem",
  "reconexion": "5s"
}
```

La sección `"catalogo"` posee los siguientes valores para conectarse:

- `"host"` tiene la ruta del catálogo de servicios. Este valor es distinto según si está usando el nodo en el ambiente de desarrollo o en el ambiente de producción.
- `"tls"` tiene las rutas de las llaves pública y privada necesarias para conectarse al catálogo de servicio de producción, ya que solo el catálogo de servicios de producción el que exige TLS mutuo para conectarse.
- El parámetro `"jwtkey"` contiene la llave pública del servidor central que sirve para validar los tokens de autorización que son emitidos por este servidor central.
- El parámetro `"autorizadorCertificadoPublico"` tiene la ruta del certificado público que permite validar los tokens del autorizador ciudadano.
- El parámetro `"certificadoPublico"` tiene la ruta del certificado público que permite identificar y validar al servidor central a la hora de conectarse mediante TLS Mutuo.
- El parámetro `"reconexion"` indica la cantidad de tiempo que espera el Nodo para intentar reconectar con el servidor central en caso de que pierda la conexión. Este valor se compone de un valor numérico y la unidad de tiempo a la que se refiere.

Ejemplo: `"5s"` cinco segundos

4.5.7. Configuración de la base de datos local

El Nodo posee una base de datos local que sirve para almacenar datos necesarios para el funcionamiento del Nodo (certificados públicos de sus contrapartes, endpoints de los servicios consumidos, traza aún no enviada, etc.)

En el atributo `"path"` se coloca la ruta donde se creará la base de datos del Nodo.

```
"database": {  
  "path": "/ruta/base_temporal_de_mi_nodo"  
}
```

NOTA

Si hay dos Nodos funcionando en la misma máquina deben tener cada uno una base de datos local diferente.

4.5.8. Configuración del Log

El Nodo de interoperabilidad posee un log que permite ver su actividad en tiempo de ejecución y es posible configurarlo a las necesidades de cada organismo.

```
"log": {  
  "nivel":      "DEBUG",  
  "ruta":       "stdout",  
  "color":      true,  
  "megabytes":  1,  
  "MaxBackups": 10,  
  "diasMaximo": 30  
}
```

- El `"nivel"` indica cual es el nivel de mensajes de logs que la aplicación Nodo va a escribir. Los posibles niveles son: `"ERROR"`, `"INFO"` y `"DEBUG"`
- La `"ruta"` indica donde se escriben los logs de la aplicación. Si se coloca el valor `"stdout"` se escribirán los mensajes en la misma salida estándar de la terminal, en cambio si se coloca cualquier otro valor el Nodo lo utilizará como una ruta de archivo y escribirá los mensajes en dicho archivo.
- El atributo `"color"` indica si los mensajes de error se escriben con diferentes colores en la interfaz o monocromáticos. Los valores permitidos son `true` y `false`.
- `"megabytes"` indica el tamaño máximo que puede alcanzar el archivo de log. Cuando se alcanza el tamaño máximo del archivo de log se convierte en un archivo de backup y se crea un nuevo archivo de log limpio.
- `"MaxBackups"` indica la cantidad máxima de archivos de backups que el Nodo guardará.
- `"diasMaximo"` indica la cantidad máxima de tiempo que se guardan los archivos de backup.

4.6. Comandos del Nodo

El Nodo se controla mediante comandos, los cuales permiten inicial, controlar y extraer información importante del Nodo en ejecución.

`./NodoV2 start` Inicia el Nodo de Interoperabilidad

```
$ ./NodoV2 start

Nodo de Interoperabilidad

Version: 2.0.8 Habilis
Started at: 16/02/2024 10:39:35

LOG LEVEL: debug
[2024-02-16 10:39:38] INFO NODO Interoperabilidad | Version: 2.0.8 Habilis | Started at: 16/02/2024 10:39:38
[2024-02-16 10:39:38] INFO Creando servidores ...
[2024-02-16 10:39:38] INFO Servidor Interno: Creando el Servidor Interno...
[2024-02-16 10:39:38] INFO [Servidor Externo] Creando el servidor externo...
[2024-02-16 10:39:38] INFO [Servidor Externo] Servidor Externo desplegado (puerto 8088)
[2024-02-16 10:39:38] INFO Conexión al servidor central: true
[2024-02-16 10:39:38] DEBUG Dirección del servidor central: CentralServerPISEEE = {0 CentralServerPISEEE 1 https://18.237.240.119:8444 [PISEEE]}
[2024-02-16 10:39:38] INFO Successfully loaded 17 local certificates.

***** EVALUACION DE ACCESOS *****

[Catalogo]
Acceso ok a 18.237.240.119:8500

[Servidor Central]
Acceso ok a 18.237.240.119:8444

*****

[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */firma*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */listadoOrganismos*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */detalleOrganismo/{codigo:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/habilidad/{rut:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/habilidad*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/habilidad_historico/{rut:.*}/{desde:.*}/{hasta:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/comp_base/{rut:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/comp_base*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/comp_evaluacion/{rut:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/comp_evaluacion*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/ventas/{rut:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/sellos/{rut:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/sellos*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */chilecompra/dj_socios/{rut:.*}*
[2024-02-16 10:39:39] INFO [Servidor Interno] - escuchando en el endpoint: */file/exchange*
[2024-02-16 10:39:39] INFO [Servidor Interno] Servidor Interno desplegado (puerto 8085)
[2024-02-16 10:39:40] INFO Conexión al Catálogo de Servicios: OK
[2024-02-16 10:39:40] INFO Conexión al Servidor Central: OK
[2024-02-16 10:39:40] INFO Id organismo: 7
[2024-02-16 10:39:40] INFO

*****
EXITO!
Conectado al Servidor Central de PISEE 2.
El NODO esta listo para ser utilizado
*****

[2024-02-16 10:39:41] DEBUG CERT: Llegaron 34 certificados
```

`./NodoV2 stop` Detiene el Nodo de Interoperabilidad

```
$ ./NodoV2 stop
Deteniendo el Nodo de Interoperabilidad ...
Nodo detenido.
$
```

```
./NodoV2 version
```

Muestra la versión del Nodo

```
$ ./NodoV2 version
Nodo de Interoperabilidad PISEE 2
Versión: 2.0.0 Habilis
```

```
./NodoV2 autoevaluacion
```

Efectúa una evaluación de accesos y permisos

```
$ ./NodoV2 autoevaluacion

+++++ EVALUACION DEL NODO +++++

[Base de datos local]
Acceso ok a Base de datos local: /tmp/desa/NODOROD

+++++ EVALUACION DE ACCESOS +++++

[Catalogo]
Acceso ok a 18.237.240.119:8500

[Servidor Central]
Acceso ok a 18.237.240.119:8444

+++++
```

```
./NodoV2 estado
```

Muestra indicadores del estado de ejecución del Nodo

```
$ ./NodoV2 estado
***** Estado del Nodo *****
Nodo Activo: OK
Codigo Organismo: 6

-----
Conectado a base de datos local: OK
Conectado al catalogo de servicios: OK
Conectado al servidor central: OK

-----
Log: DEBUG ruta: stdout

-----
Puerto Interno: 8084
Puerto Externo: 8489
*****
```

```
./NodoV2 certificados
```

Lista todos los certificados cargados

```
$ ./NodoV2 certificados

CERTIFICADOS CARGADOS
[ID] Organismo Firma TLS
[ 0] 100 ok ok
[ 1] 1000 ok ok
[ 2] 1003 ok ok
[ 3] 1010 ok ok
[ 4] 1015 ok ok
[ 5] 1016 ok ok
[ 6] 15 ok ok
[ 7] 2 ok ok
[ 8] 5 ok ok
[ 9] 7 ok ok
[10] 8 ok ok
```



```
./NodoV2 certificados <id_organismo>
```

Muestra el detalle de los certificados de un organismo

```
$ ./NodoV2 certificados 100
***** Certificados del Organismo: 100 *****
Certificado de Firma:
d766a961fde99b7f98e38903f901dd7c9fe867567b752eb6888d59207a8403a7

Certificado de TLS:
-----BEGIN CERTIFICATE-----
MIIDljCCAN6gAwIBAgIUaRYLIuHLws+8U181B9WBSPTYG09IwDQYJKoZIhvcNAQEL
BQAwSTELMAKGA1UEBhMCQ0wxZzAJBgNVBAGMALJNMRYwFAYDVQQKDA1Fc3RhZG9E
ZUNoaWxMRUwEwYDVQQDDAxFSkVNUExPXM05PRE8wHhcNMjAwODI0MTYyMzA0WWhcN
MzAwODIyMTYyMzA0WjBjMQswCQYDVQQGEwJDTDELMAKGA1UECAwCUk0xZjAUBGNV
BAoMDUVzdGFkb0RlQ2hpbGUXFTATBgNVBAMMDEVKRU1QTE9fTk9ETzCCASiWdQYJ
KoZIhvcNAQEBBQADggEPADCCAQoCggEBAM9249D3LknzZ8He3Mj01m6WLHei11zh
k0iUmEnHXeg+Y1M6oSNVxNMLcgLjdEt6WKWAFHHyGbTpJ26idDY+MJ0hnp133SC5
jCBisgErm9mnzEDIpVLG893jIz8FZq9AuIQFAUsNbZHN+Bps0wIVOfi7gFKtTUqX
0s3FaZ9za39B8TmZiJPLncoAhdKIjX31eNfVh9aKkyEOD6Kkzd7HpJtIAD9NZNaE
k/QSV/h2hX401C+R6wT8Fbub4SeW7c7RCbQayI3fM4jax9Gjz83VpHKB338TencK
8bJTOqyhZuussY8B8rEyPnHOjujh/4fv2unNEPgCbzmNrsI6T0QLM1cCAwEAAaN2
MHQWHQYDVR00BBYEFNFON8bY0pWJjLyOmLRWeSStW4SRMB8GA1UdIwQYMBaAFNFO
N8bY0pWJjLyOmLRWeSStW4SRMA8GA1UdEwEB/wQFMAMBAf8wDwYDVR0RBAGwBocE
Eu3wdzAQBgNVHSAECTAHMAUGAyoDBDANBgkqhkiG9w0BAQsFAAOCQAQEAy2J1awUX
7PEVIFbWCT+Y5FtiEV6zUyp/J70DFmy1/ENDxv7LFLoQg1yzG8A/eSG5nLgdeB9b
j1xVAAGwZrn3s/9GpZ1X73wnWl/22jDQpxbu3k/CHMcXSKNZIU+KUC15vUZKdCJ
Oh+AhaAYBPbyiuLqSQWh36b4ZFE6hays3fv7/tAGn80pmepIiNfhwqF/6NbnbmTN
FIDf55Aj4IvWTLz3ySMjAT7RkIwHpgjVlw3GASxl0xFDpTMwU5N8H0qV9TjnE0wY
op4UMg3pIx0y5mFHLkBVkQuSs6PXHTb7L204wTsujtY9i52svIw8ax2CseGasjux
EDa3B1rzb5DxoA==
-----END CERTIFICATE-----
```

```
./NodoV2 certificados --descarga
```

Obliga al nodo a efectuar una recarga forzada de los certificados

```
$ ./NodoV2 certificados --descarga
Solicitando descarga... ForceDownload

Carga finalizada.
```

`./NodoV2 servicios`

Lista los servicios configurados

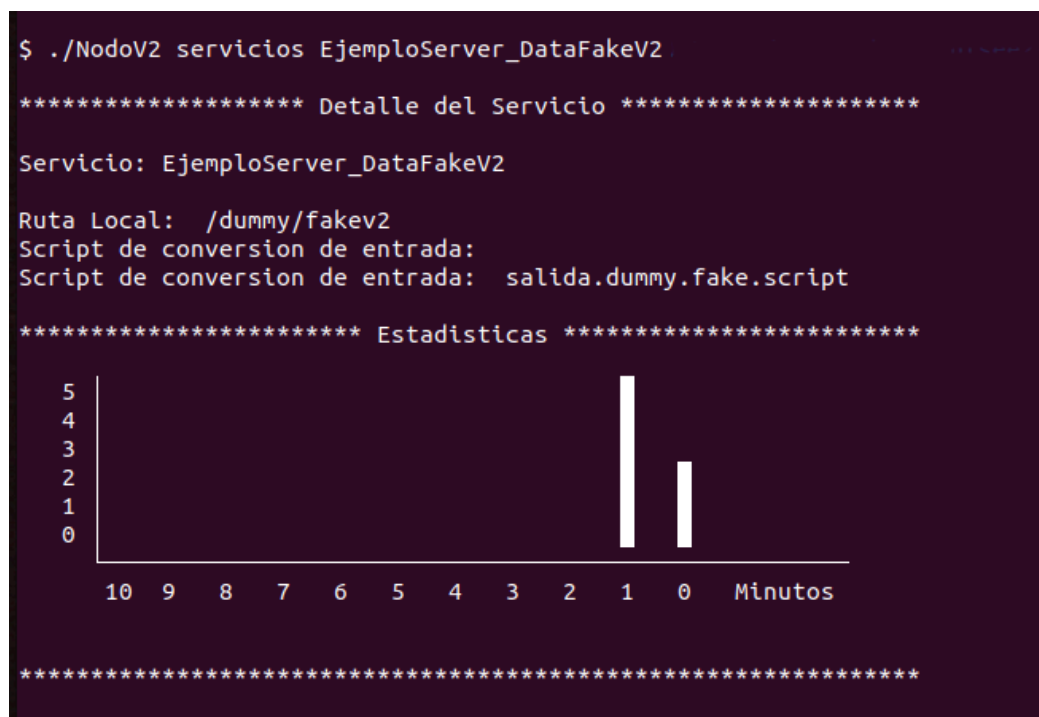
```
jcd@ $ ./NodoV2 servicios
```

SERVICIOS QUE SE CONSUMEN				
[ID]	Nombre	Ruta Local	Conversion de entrada	Conversion de salida
[0]	Accesspoint	/accesspoint/newmodel		
[1]	Accesspoint2	/accesspoint/newmodel_doc		
[2]	Accesspoint3	/accesspoint/newmodel_get		
[3]	AccesspointDoc	/accesspoint/newmodel_getdoc		
[4]	plseespdip	/datosSRCEI/test		
[5]	EstadoDeudaPorRol	/accesspoint/tgr_rol/{comuna:[0-9]+}/bienes-raices/{rol}		
[6]	EstadoDeudaPorRol	/accesspoint/tgr_comunas		
[7]	ValorPropiedad	/sll/valProp		
[8]	ValidaBeneficiarioCapredena	/accespoint/capredena		
[9]	datosContribuyente	/sll/datosContribuyente		
[10]	ServicioMinvuPruebaProveedor	/minvu/prueba		
[11]	BeneficiosPotenciales	/nds/beneficios		
[12]	busquedaDocumentosDocDigital	/docdigital/buscar		
[13]	SRCEIInformacionPersonalDomicilioMTTPProxy	/SRCEI/datosPersonalDomicilio		
[14]	SRCEIInformacionLicenciasMTTPProxy	/SRCEI/informacionLicencias		
[15]	SIIComunaRun	/sll/getComuna		
[16]	DatosIndividuo	/srcei/individuo		
[17]	EjemploServer_Echo	/dummy/echo	entrada.dummy.echo.script	
[18]	EjemploServer_DataFake	/dummy/fake		salida.dummy.fake.script
[19]	EjemploServer_DataFakeV2	/dummy/fakev2		salida.dummy.fake.script
[20]	Excel_test	/dummy/excelv2		

SERVICIOS QUE SE PROVEEN					
[ID]	Nombre	Ruta externa	TPS	tipo	Ruta Local
[0]	AccesspointIPjcd	/wena	150000	WEB SERVICE	http://localhost:8092/ejemploJson
[1]	ExcelDeEjemplo	/elexcel	10	EXCEL	
[2]	AccesspointIPjcdFull	/hello	150000		
[3]	AccesspointIPjcdGet	/helloget	7000		
[4]	DatosPersonaSRCEIjcd	/datosPersonaSRCEI	10		
[5]	AccesspointIPjcdExchangeFile	/exchangeFile	2		

`./NodoV2 servicios <nombre_servicio>`

Muestra el detalle de un servicio y sus estadísticas de uso más reciente



```
./NodoV2 trazabilidad
```

Muestra estadísticas de uso del Nodo de interoperabilidad en tiempo de ejecución.

`./NodoV2 clean` Limpia la base de datos local del Nodo, lo cual obliga al Nodo a volver a cargar los certificados de las contrapartes y los endpoints de los servicios con los cuales interopera.

```
$ ./NodoV2 clean
Base de datos del nodo 100% limpia.
```

Tabla I - Resumen de comandos nodo

Comando	Descripción
<code>./NodoV2 start</code>	Inicia el Nodo de Interoperabilidad
<code>./NodoV2 stop</code>	Detiene el Nodo de Interoperabilidad
<code>./NodoV2 version</code>	Muestra la versión del Nodo
<code>./NodoV2 autoevaluacion</code>	Efectúa una evaluación de accesos y permisos
<code>./NodoV2 estado</code>	Muestra indicadores del estado de ejecución del Nodo
<code>./NodoV2 certificados</code>	Lista todos los certificados cargados
<code>./NodoV2 certificados <id_organismo></code>	Muestra el detalle de los certificados de un organismo
<code>./NodoV2 certificados --descarga</code>	Obliga al nodo a efectuar una recarga forzada de los certificados
<code>./NodoV2 servicios</code>	Lista los servicios configurados
<code>./NodoV2 servicios <nombre_servicio></code>	Muestra el detalle de un servicio y sus estadísticas de uso más reciente
<code>./NodoV2 trazabilidad</code>	
<code>./NodoV2 clean</code>	Limpia la base de datos local del Nodo, lo cual obliga al Nodo a volver a cargar los certificados de las contrapartes y los endpoints de los servicios con los cuales interopera.

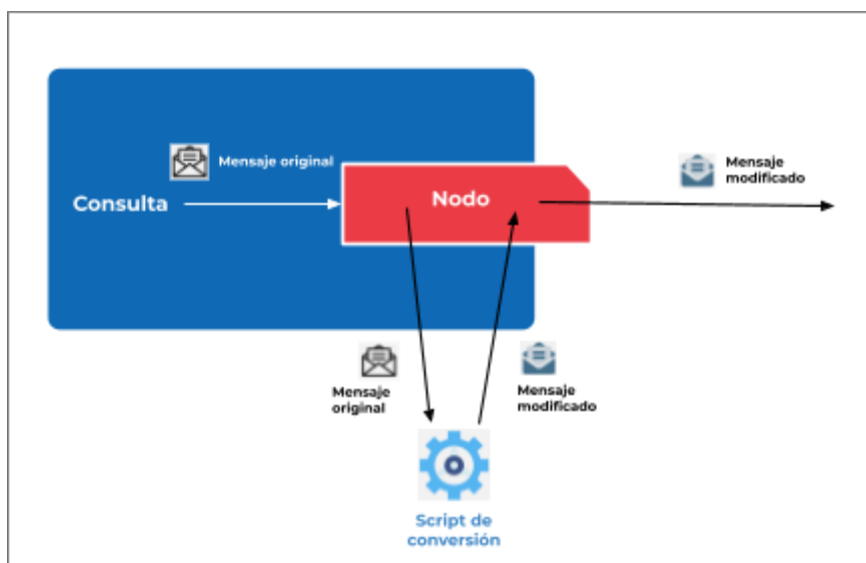
5. Conversión de mensajes

5.1. Concepto

Desde la versión 2 del Nodo es posible convertir los mensajes de entrada y salida de forma personalizada mediante script asociados a cada servicio que se consume.

Esto se logra gracias a que el Nodo, antes de enviar los datos al otro organismo, ejecuta un script que recibe el mensaje original, lo convierte a lo que requiera y se lo devuelve al Nodo que ahora, ya convertido, lo envía al otro organismo.

El mismo flujo, a la inversa, se produce con la respuesta que entrega el otro organismo.



Conversión de mensajes a enviar

Esta funcionalidad proporciona varios beneficios:

- Permite adaptar los mensajes a lo que requieren los aplicativos existentes sin tener que modificar dichos aplicativos.
- Al estar los script desacoplados del Nodo es posible modificarlos según las necesidades de cada organismo.

- Se pueden modificar los mensajes que se envían, las respuestas o ambos. Dependiendo de las necesidades de cada organismo.

5.2. Cómo convertir mensajes para un servicio

5.2.1. Asignar un script al mensaje de consulta

Para agregar un script al mensaje que se va a enviar a través del Nodo se debe colocar la siguiente configuración en el archivo `config.json`.

```
{
  "nombre": "NombreDelServicio",
  "rutaLocal": "/ruta/local/servicio",
  "conversion": {
    "entrada": "nombre_de_archivo.script"
  }
}
```

Donde el valor de `"entrada"` corresponde al nombre del archivo de script a ejecutar, el cual debe encontrarse en la carpeta `./conversiones` del Nodo.

5.2.2. Asignar un script al mensaje de respuesta

Para agregar un script al mensaje que se va a recibir a través del Nodo se debe colocar la siguiente configuración en el archivo `config.json`.

```
{
  "nombre": "NombreDelServicio",
  "rutaLocal": "/ruta/local/servicio",
  "conversion": {
    "respuesta": "nombre_de_archivo.script"
  }
}
```

Donde el valor de `"respuesta"` corresponde al nombre del archivo de script a ejecutar, el cual debe encontrarse en la carpeta `./conversiones` del Nodo.

5.3. Cómo escribir los script de conversión

Los script de conversión deben ser un único archivo con código fuente escrito en lenguaje **Go Lang** (<https://go.dev/doc/>). Este archivo debe tener una estructura particular y se comunica con el Nodo mediante un objeto que comparten ambos componentes y funciona como medio de comunicación.

La estructura que debe respetar el script es la siguiente:

```
package main

import (
    "Nodo"
    "time"
    "fmt"
    "encoding/xml"
    // Colocar aqui las librerias necesarias
)

type ... struct . . .

func main () { . . .

func convertirEnvio(){ . . .

func convertirRespuesta(){ . . .
```

- Primero define el package.
- Importa las librerías necesarias.
- Define estructuras específicas para convertir los mensajes. Esta sección debe ser modificada para .
- Luego tiene la función principal, `main`, que solo hace la derivación según si el mensaje es de envío o de respuesta.
- Al final vienen las dos funciones de conversión y son estas funciones `convertirEnvio` y `convertirRespuesta`.

En la sección `import` se debe importar la librería `"Nodo"` que contiene el objeto `Nodo` que sirve para la comunicación entre el `Nodo` y el script, solo a través de este objeto se efectúa la comunicación entre ambas partes, usando sus campos y sus métodos.

Este objeto tiene una serie de campos y métodos:

```
type TypeNodoChannel struct {
    prefijoLog      string
    Accion          string
    MensajeOriginal string
    MensajeConvertido string
    Error           error
}
```

<code>prefijoLog</code>	tiene el ID único de la transacción
<code>Accion</code>	Indica si es un <code>"ENVIO"</code> o una <code>"RESPUESTA"</code>
<code>MensajeOriginal</code>	contiene el mensaje original como un string
<code>MensajeConvertido</code>	aquí el script debe colocar el mensaje convertido
<code>Error</code>	Sirve para informar al <code>Nodo</code> de un error en la conversión

`WriteLog()` es el método que permite al script escribir mensajes en el log del nodo.

Para utilizar este objeto se debe llamar con `Nodo.Channel.<campo o metodo>`.
Ejemplo:

```
Nodo.Channel.MensajeConvertido = "mensaje convertido"

o

Nodo.Channel.WriteLog( "algo" )
```

En este ejemplo se puede ver como funciona un script de conversión, que cambia un mensaje XML en un JSON:

```
package main

import(
    "Nodo"
    "time"
    "fmt"
    "encoding/xml"
)

const (
    ENVIO string = "ENVIO"
    RESPUESTA = "RESPUESTA"
)

type Nota struct {
    XMLName xml.Name `xml:"note" json:"- "`
    To string `xml:"to" json:"destinatario"`
    From string `xml:"from" json:"desde"`
    Heading string `xml:"heading" json:"cabecera"`
    Body string `xml:"body" json:"cuerpo"`
}

func main() {
    startAll := time.Now().UnixNano() / int64(time.Millisecond)
    switch Nodo.Channel.Accion {
    case ENVIO:
        convertirEnvio()
    case RESPUESTA:
        convertirRespuesta()
    default:
    }

    Nodo.Channel.WriteLog( fmt.Sprintf("Duration(ms) Total Scriggo: %d",
    (time.Now().UnixNano() / int64(time.Millisecond)) - startAll ))
}
```



```
}

/**
 * Convierte el mensaje de Envio
 */
func convertirEnvio() {
    Nodo.Channel.WriteLog("Ejecutando SCRIPT envio: " +
Nodo.Channel.MensajeOriginal)
    v := Nota{}
    err1 := xml.Unmarshal([]byte(Nodo.Channel.MensajeOriginal), &v)
    if err1 != nil {
        Nodo.Channel.WriteLog( fmt.Sprintf("error: %v", err1))
        Nodo.Channel.Error = err1
        return
    }
    s := fmt.Sprintf( `{'destinatario': '%s', 'desde': '%s',
'cabecera': '%s', 'cuerpo': '%s'}`, v.To, v.From, v.Heading, v.Body )

    Nodo.Channel.MensajeConvertido = s
    Nodo.Channel.Error = nil
}

/**
 * Convierte el mensaje de Respuesta
 */
func convertirRespuesta() {
    Nodo.Channel.WriteLog("Ejecutando SCRIPT respuesta, con valor: " +
Nodo.Channel.MensajeOriginal)

    v := Nota{}
    err1 := xml.Unmarshal([]byte(Nodo.Channel.MensajeOriginal), &v)
    if err1 != nil {
        Nodo.Channel.WriteLog( fmt.Sprintf("error: %v", err1))
        Nodo.Channel.Error = err1
        return
    }
}
```

```
s := fmt.Sprintf( `{"destinatario":"%s", "desde":"%s",
"cabecera":"%s", "cuerpo":"%s"}`, v.To, v.From, v.Hheading, v.Body )

Nodo.Channel.MensajeConvertido = s
Nodo.Channel.Error = nil
}
```

Este código se almacena en un archivo y en el archivo `config.json` se hace la referencia a que será utilizado para convertir mensajes de un servicio en particular.

6. Interfaces de consulta

6.1. Concepto

El Nodo de Interoperabilidad se crea para dar respuesta a las necesidades de interoperar datos entre organismos públicos a través de web services de manera segura, pero eso requiere que el organismo consumidor tenga una aplicación que pueda realizar esos llamados a servicios web.

Para aquellos organismos públicos, o áreas de algún organismo público, que no posean una aplicación que haga esas consultas tanto por temas presupuestarios, técnicos o porque el volumen de consultas no lo justifica el Nodo, desde su versión 2.0, tiene la posibilidad de agregar interfaces web asociadas a los servicios web lo que permite a usuarios no técnicos utilizar los servicios de interoperabilidad al cual le han otorgado acceso.

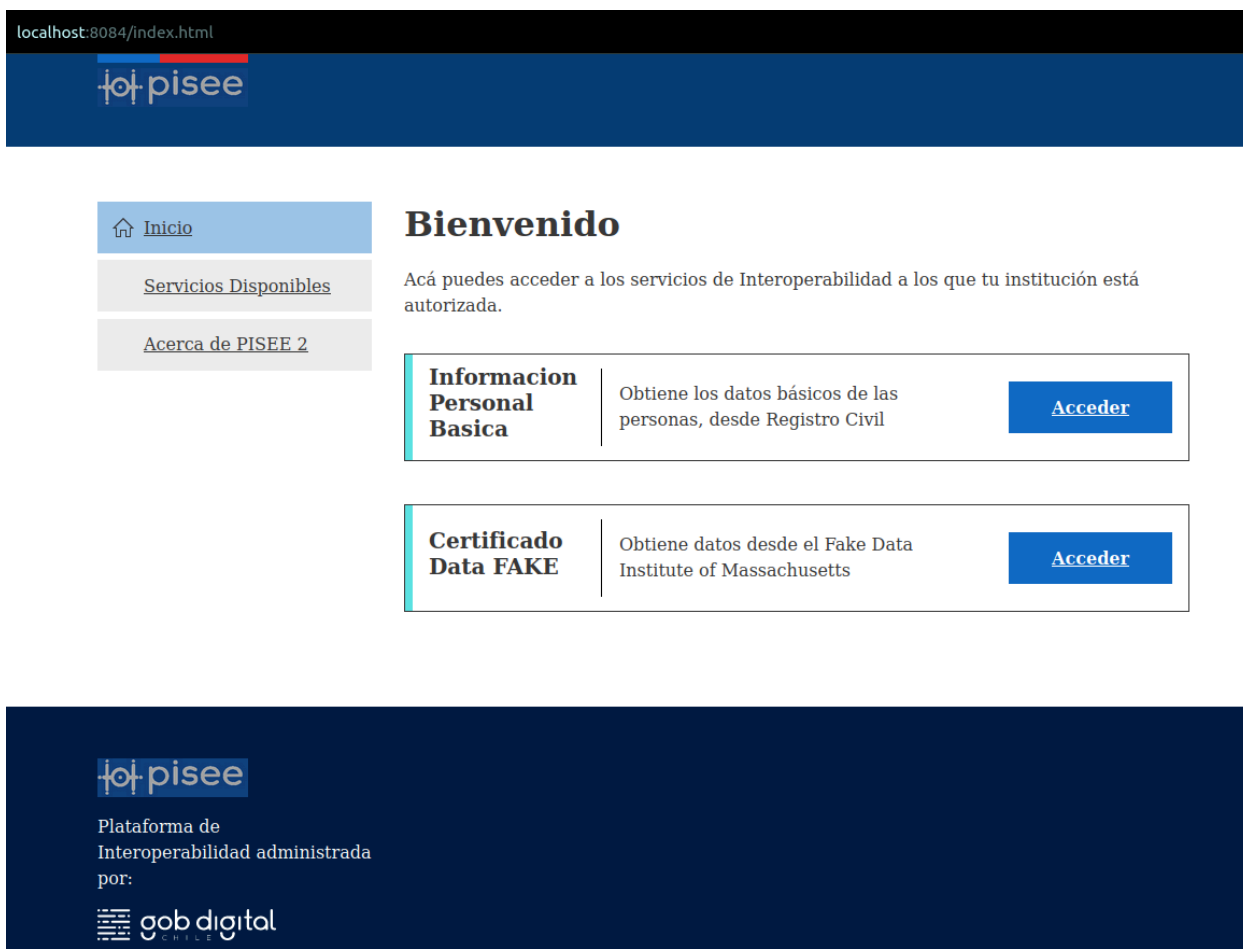
Estas interfaces corresponden a páginas web que el mismo nodo publica localmente y que se pueden acceder desde la interfaz del Nodo. La tecnología de estas interfaces es HTML+VueJS+CSS que son las tecnologías más populares en la programación web y permitirán que tanto los organismos proveedores como los consumidores puedan adaptar las interfaces a sus necesidades.

6.2. Cómo utilizar las interfaces

En un navegador web entre a:

`http://<ip_del_nodo>:<puerto_del_nodo>/index.html`

(Ejemplo: <http://localhost:8084/index.html>) y aparecerá la siguiente pantalla:



En esta pantalla aparece el listado de todas las interfaces que el Nodo encuentre dentro de la carpeta `./assets` del mismo Nodo.

Al hacer click en el botón “Acceder” de una de estas interfaces mostrará la interfaz seleccionada.

En este caso se muestra una interfaz de prueba que consulta datos básicos de registro civil:



[Volver](#)

Servicio de consulta de Informacion personal basico

Ingrese el RUN de la persona a consultar:

RUN:

DV:

[Consultar](#)

Si ingresa los datos y presiona consultar, la interfaz realizará la consulta HTTP al Nodo, el Nodo enviará la consulta al proveedor del servicio y luego obtendrá los datos de la respuesta. Cuando el Nodo reciba los datos se los entregará a la interfaz para que esta los despliegue tal como ha sido diseñada. En este ejemplo simplemente extrae los datos y los muestra abajo.



[Volver](#)

Servicio de consulta de Informacion personal basico

Ingrese el RUN de la persona a consultar:

RUN:

DV:

[Consultar](#)

Datos personales

Nombre: JUAN CARLOS

Apellido Paterno: DAILLE

Apellido Materno: PALACIOS

Fecha nacimiento: 1982-03-13

Estado civil: S

Nacionalidad: C

6.3. Cómo asignar una interfaz para un servicio

No es necesario realizar ninguna acción en el archivo `config.json` para asignar una interfaz a un servicio ya que el Nodo busca automáticamente en la carpeta `./assets` todos los archivos de interfaces y los vincula a los servicios configurados en el `config.json`, por esa razón es muy importante que estén bien escritas las cabeceras en el archivo HTML de la interfaz, especialmente el nombre de servicio para que pueda hacer el vínculo.

6.4. Cómo escribir una interfaz

Escribir un archivo de interfaz se reduce a escribir el contenido del cuerpo del sitio en HTML con código controlador hecho en Javascript, específicamente con la librería VueJS (<https://v2.vuejs.org/v2/guide/>).

localhost:8084/interfaces/informacion_basica.html

topisee NODO Interoperabilidad

[Resolver](#)

Servicio de consulta de Información personal básico

Ingrese el RUN de la persona a consultar:

RUN:

DV:

Consultar

Datos personales

Nombre: JUAN CARLOS

Apellido Paterno: DAILLE

Apellido Materno: PALACIOS

Fecha nacimiento: 1982-03-13

Estado civil: S

Nacionalidad: C

Cuerpo

La interfaz posee la siguiente estructura:

```
<!-- NOMBRE: SRCeIInformacionPersonalDomicilioMTTProxy -->
```

```

<!-- TITULO: Informacion Personal Basica -->
<!-- DESCRIPCION: Obtiene los datos básicos de las personas, desde el Registro Civil -->

<div class="d-none d-lg-block col-lg-12" id="app">
    Aqui va el contenido del HTML
</div>
<script>
    Aqui va el controlador javascript
</script>

```

Primero para que el Nodo entienda a qué servicio corresponde dicha interfaz es necesario colocar una cabecera en las primeras líneas del archivo html.

- El NOMBRE debe ser el mismo del catálogo de servicios y que aparece en el archivo `config.json`
- TITULO es el título que aparecerá en la interfaz principal donde aparece la lista de interfaces
- DESCRIPCION es la descripción larga del servicio que aparecerá en la interfaz principal donde aparece la lista de interfaces

Luego se coloca el HTML que define los componentes de la interfaz:

- Dentro del `<DIV>` va el contenido HTML de la interfaz

Y por último va el código javascript que controla el comportamiento de la interfaz y para el cual se puede utilizar VueJS y Axios ya que ambas librerías están precargadas en el sitio web:

- Dentro de `<SCRIPT>` va el controlador Javascript de la interfaz

7. Consideraciones Generales

7.1. Transacciones

- Cada proveedor fija la cantidad de transacciones que se pueden realizar en un periodo de tiempo. Por ejemplo, el Registro Civil solo permite un máximo de 2 transacciones por segundo (TPS).
- El Nodo permite autorregular la cantidad de transacciones que se realizan. Esto se hace en el archivo de configuración.

7.2. Política de actualizaciones

- DGD creará una plataforma web para la red de interoperabilidad del Estado y ahí se encontrarán todas las versiones del Nodo, con las características de cada cual.
- Las versiones del Nodo será retrocompatibles en funcionamiento con las anteriores, para no afectar la operación
- El código fuente del Nodo será liberado bajo licencia GNU GENERAL PUBLIC LICENSE Versión 3 y se disponibilizará el repositorio en GIT. Las instituciones serán libres de ocupar la rama principal o de modificarla a su discreción.

8. Anexos

8.1. Obtener llaves pública y privada desde un .pfx

- Pública:

```
openssl pkcs12 -in <nombre_archivo>.pfx -clcerts -nokeys -out  
public.pem
```

- Privada:

```
openssl pkcs12 -in <nombre_archivo>.pfx -nocerts -nodes -out  
private.pem
```

8.2. DockerFile de Ejemplo

```
FROM ubuntu:20.04  
  
WORKDIR /home/Nodo  
RUN apt-get update && apt-get -y install xmlsec1  
COPY . /home/Nodo/  
  
EXPOSE 8084  
RUN chmod +x ./NodoV2  
  
CMD [ "./NodoV2", "start"]
```